



**université
virtuelle**
Burkina ★ Faso



**Sciences
du Numérique**

Théorie des Langages et Compilation MSI-02

Analyse Lexicale



Dr. Tegawendé F. BISSYANDE
tegawende.bissyande@gmail.com

Semestre 1

“Sécurité des Réseaux et des Logiciels”

Optimization / Code

Analyse Sémantique

Analyse Syntaxique

Analyse Lexicale

Intro

Analyse Lexicale: Kesako?

- Objectifs
 - Lire les caractères du programme source
 - Produire un flot de mots (lexèmes)
- Crible
 - La suite de lexèmes est ensuite filtré pour produire une représentation interne : les unités lexicales
- Relation avec la phase d'analyse syntaxique
 - Les unités lexicales sont les données d'entrée de l'analyseur syntaxique

Bases théoriques

- Langages réguliers
- Expressions régulières
- Automates finis
 - Automate fini non déterministe (NFA)
 - Automate fini déterministe (DFA)
 - Transformation d'un NFA en DFA
 - Minimisation du nombre d'états d'un DFA
- Construction d'un AFN à partir d'une expression régulière

Théorie des Langages

Fondamentaux d'un langage

- Alphabet
 - Un **alphabet** Σ est un ensemble fini et non vide de caractères
 - Exemple : $\Sigma = \{a, b\}$
- Mot
 - Un **mot** (aussi appelé une chaîne), ω sur Σ , est une suite finie de caractères de Σ juxtaposés
 - Exemple : $\omega = abaab$
 - Le mot vide, ne contenant aucun caractère, est noté ϵ
Attention : ϵ peut-être un mot, mais n'est pas une lettre de l'alphabet

Définition d'un mot

- Définition

- $|\omega|$ dénote la longueur du mot ω , c'est à dire le nombre de caractères contenus dans ω
- $|\omega|_\alpha$ dénote le nombre d'occurrences de la lettre $\alpha \in \Sigma$ dans le mot ω

- Exemples

$$\Sigma = \{a, b\}$$

$\omega = abaab$ est un mot sur Σ

$$|\omega| = 5$$

$$|\omega|_a = 3$$

Notations

- Ensemble des mots sur Σ de longueur nulle

$$\Sigma^0 = \{\epsilon\}$$

- Ensemble des mots sur Σ de longueur n

$$\Sigma^n$$

- Ensemble de tous les mots sur Σ

$$\Sigma^* = \bigcup_{n \geq 0} \Sigma^n$$

- Ensemble des mots non vides sur Σ

$$\Sigma^+ = \bigcup_{n \geq 1} \Sigma^n = \Sigma^* - \epsilon$$

Concaténation de deux mots

- Définition

- Si x et y sont deux mots sur Σ , on appelle **concaténation** de x et y le mot $x.y$

- Exemples

$$x = ab, y = cd$$

$$x.y = abcd$$

- Remarque

$$\text{On note également } x.y = xy = x \circ y$$

- Autre opérations de concaténation

Concaténation avec le mot vide : $\alpha.\epsilon = \epsilon.\alpha = \alpha$

On note a^n la concaténation de n occurrences de a

a^0 dénote le mode vide ϵ

Fermeture de Kleene

Soit Σ un alphabet, on appelle **fermeture de Kleene**, de Σ , noté Σ^* , l'ensemble défini inductivement de la façon suivante :

- Base : tous les caractères de Σ ainsi que le mot vide ϵ sont dans Σ^*
- Règle : si x et y sont dans Σ^* , alors $x.y$ est dans Σ^*

Rappels

$$\Sigma^* = \bigcup_{n \geq 0} \Sigma^n$$

$$\Sigma^+ = \bigcup_{n \geq 1} \Sigma^n = \Sigma^* - \epsilon$$

Langages

- Définition
 - On appelle **langage formel** (souvent noté L) sur un alphabet Σ un sous ensemble de Σ^*
- Union et intersection de deux langages

Soient L_1 et L_2 deux langages sur l'alphabet Σ

$$L_1 \cup L_2 = \{\omega \mid \omega \in L_1 \text{ ou } \omega \in L_2\}$$

$$L_1 \cap L_2 = \{\omega \mid \omega \in L_1 \text{ et } \omega \in L_2\}$$

Opérations sur les langages (1)

- Produit de deux langages

Soient L_1 un langage sur l'alphabet Σ_1 et L_2 un langage sur l'alphabet Σ_2

$$L_1 L_2 = \{\omega_1 \omega_2 \in (\Sigma_1 \cup \Sigma_2)^* \mid \omega_1 \in L_1 \text{ et } \omega_2 \in L_2\}$$

- Notations

$$L^0 = \{\epsilon\}$$

$$L^n = L L^{n-1}, \forall n \geq 1 = \{\omega_1 \omega_2 \dots \omega_n \mid \omega_i \in L_i, 1 \leq i \leq n\}$$

Opérations sur les langages (2)

- Fermeture de Kleene d'un langage

$$L^* = \bigcup_{n \geq 0} L^n$$

$$L^+ = L^* - \{\epsilon\}$$

- Complément d'un langage

On note $\bar{L}$ l'ensemble de tous les mots de Σ^* qui ne sont pas dans L

$$\bar{L} = \Sigma^* - L$$

Langages réguliers (/ rationnels)

- Définition (inductive)
 - Base : $\emptyset$, $\{\epsilon\}$ et $\{a\}$ ($\forall a \in \Sigma$) sont des langages réguliers sur Σ
 - Règle : Si R_1 et R_2 sont des langages régulières sur Σ , il en est de même pour $R_1 \cup R_2$, $R_1 R_2$ et R_1^*
- Remarque

Il n'existe pas d'autre langage régulier sur Σ

Expressions Régulières

- Définition (inductive)
 - Base : $\emptyset$, ϵ et $a \in \Sigma$ sont des expressions régulières sur Σ , représentant respectivement les langages $\emptyset$, $\{\epsilon\}$, et $\{a\}$
 - Règle : Si r_1 et r_2 sont des expressions régulières sur Σ , décrivant respectivement les langages R_1 et R_2 , alors $(r_1|r_2)$, (r_1r_2) et $(r_1)^*$ sont des expressions régulières sur Σ décrivant respectivement les langages $R_1 \cup R_2$, R_1R_2 et R_1^*
- Remarque

Il n'existe pas d'autre expression régulière sur Σ

Exemple d'expressions régulières

Expression régulière	Langage régulier décrit	Élément du langage régulier
$a b$	$\{a, b\}$	a, b
ab^*a	$\{a\}\{b\}^*\{a\}$	$aa, aba, abba, abbba, \dots$
$(ab)^*$	$\{ab\}^*$	$\epsilon, ab, abab, \dots$
$abba$	$\{abba\}$	$abba$

Remarque

$a^n b^n$ n'est pas une expression régulière

Propriétés algébriques des expressions régulières

Axiome	Description
$r s = s r$	$ $ est commutatif
$r (s t) = (r s) t$	$ $ est associatif
$(rs)t = r(st)$	la concaténation est associative
$r(s t) = rs rt$ $(r s)t = rs rt$	la concaténation est distributive par rapport à $ $
$\epsilon r = r$ $r\epsilon = r$	ϵ est l'élément neutre pour la concaténation
$r^* = (r \epsilon)^*$	relation entre $*$ et ϵ
$r^{**} = r^*$	$*$ est idempotent

Notations abrégées

- Au moins une instance

$$r^+ = rr^*$$

- Zéro ou une instance

$$r? = r|\epsilon$$

- Classe de caractères

$$[abc] = a|b|c$$

$$[a - z] = a|b|\dots|z$$

Exemple

Les identificateurs du langage C sont les mots engendrés par l'expression régulière **ident** définie ci-dessous

chiffre $\rightarrow$ $[0 - 9]$

lettre $\rightarrow$ $[a - zA - Z]$

ident $\rightarrow$ $(\text{letter}|_)(\text{letter}|\text{decdigit}|_)^*$

Automates

Automate d'états finis (FA)

- Un automate d'états finis est représenté par un graphe orienté (fini) dont les arcs sont étiquetés. Il est composé de :
 - Un nombre fini d'états Q , qui sont les sommets du graphe
 - Un alphabet Σ
 - Un ensemble fini δ de transitions, qui sont les arcs du graphe, étiquetés chacun par une (ou plusieurs) lettre(s) de Σ ou ϵ
- Parmi les états de Q , on distingue :
 - L'état initial q_0 (il y a un et un seul état initial)
 - Les états finaux (il peut y avoir plusieurs ou même aucun état final), qui constitue l'ensemble F
- Formellement, un automate d'états finis est un quintuplet $A = (Q, \Sigma, \delta, q_0, F)$

Automate d'états finis non déterministe (NFA)

- Remarque

Les arcs d'un automate d'états finis non déterministe étiquetés par le mot vide ϵ sont appelés **ϵ -transitions**.

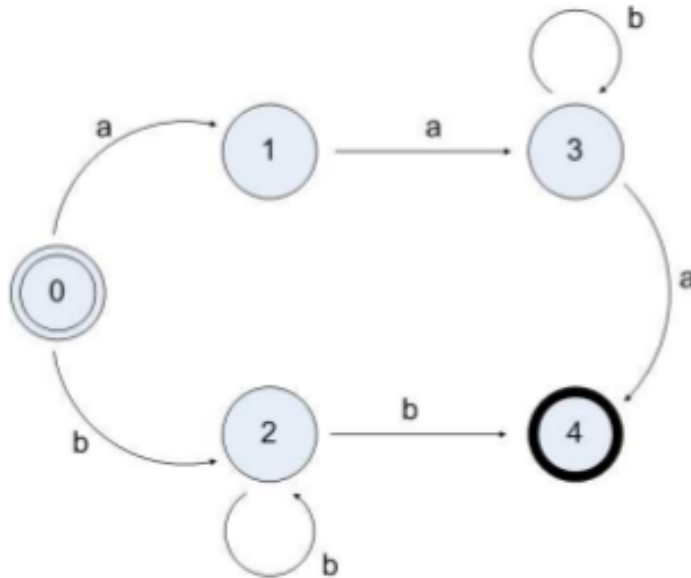
- Définition

On dit qu'un automate d'états finis $A = (Q, \Sigma, \delta, q_0, F)$ **accepte** un mot ω de Σ^* si et seulement si il existe (au moins) un chemin dans A allant de q_0 à un état final de, étiqueté par les lettres successives de ω , entre lesquelles on a éventuellement intercalé des occurrences de ϵ .

Le langage $L(A)$ reconnu par A est l'ensemble des mots que A accepte

Exemple de NFA

Automate A_1



Langage reconnu : $L(A_1) = aab^*a|bb^*b$

Table de transitions

- Définition

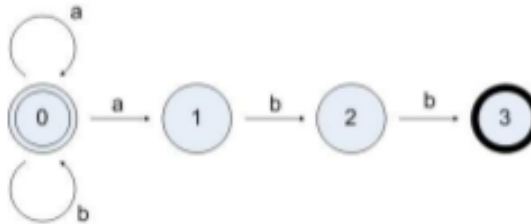
La table de transitions d'un NFA $A = (Q, \Sigma, \delta, q_0, F)$ définit la fonction δ de la manière suivante :

- Exemple

	a	b
q_0	$\{q_1\}$	$\{q_2\}$
q_1	$\{q_3\}$	$\emptyset$
q_2	$\emptyset$	$\{q_2, q_4\}$
q_3	$\{q_4\}$	$\{q_3\}$
q_4	$\emptyset$	$\emptyset$

Exemple de NFA (2)

Automate A_2



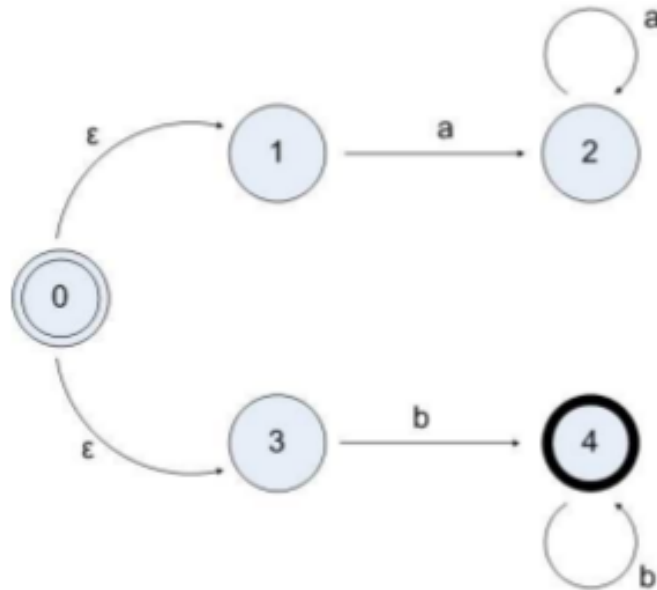
Langage reconnu : $L(A_2) = (a|b)^*abb$

Table de transitions

	a	b
0	$\{0, 1\}$	$\{0\}$
1	$\emptyset$	$\{2\}$
2	$\emptyset$	$\{3\}$

Exemple de NFA (3)

Automate A_3



Langage reconnu : $L(A_3) = aa^*|bb^*$

Automates d'états finis déterministe (DFA)

- Constat

Dans A_1 , comment interpréter $\delta(q_2, b)$? Le choix est en fait non déterministe.

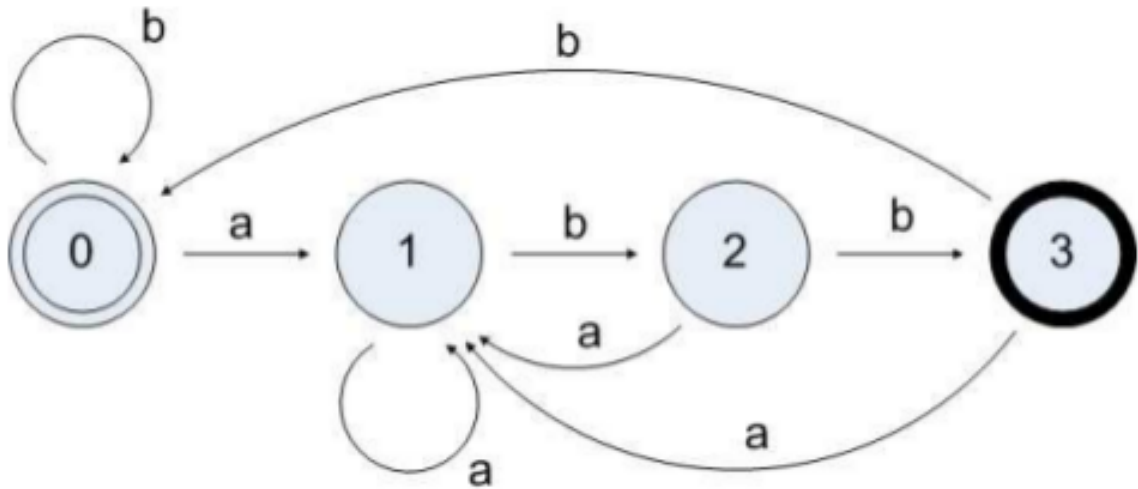
- Définition

Un automate fini déterministe (DFA) est un cas particulier d'automate fini non déterministe dans lequel :

1. Aucun état n'a de ϵ -transition, c'est à dire de transition sur l'étiquette ϵ
2. Pour chaque état e et chaque lettre a , il y a au plus un arc étiqueté a qui quitte e

Exemple de DFA

Automate A_4



Langage reconnu : $L(A_4) = (a|b)^*abb = L(A_2)$

Transformation d'un NFA en DFA

- Principe

Entrée : un AFN $N = (Q, \Sigma, \delta, q_0, F)$

Sortie : un DFA $D = (Q', \Sigma, \delta', q'_0, F')$

- Méthode

On construit la table de transition correspondant à la fonction δ' de D . Chaque état de l'AFD est un ensemble d'états de l'AFN. δ' simule en *parallèle* tous les déplacements possibles que N peut effectuer sur un mot d'entrée donnée.

Opérations sur les états d'un AFN

- Définition

On appelle ϵ -**fermeture** d'un état e , l'ensemble des états de l'AFN accessibles depuis l'état e uniquement par des ϵ -transitions

- Généralisation

On définit l' ϵ -**fermeture** d'un ensemble Q d'états par l'union des ϵ -fermetures des états appartenant à Q

- Transition

Transiter(E, x) est l'ensemble des états vers lesquels il existe une transition sur la lettre x à partir d'un état appartenant à l'ensemble E

Algorithme de détermination

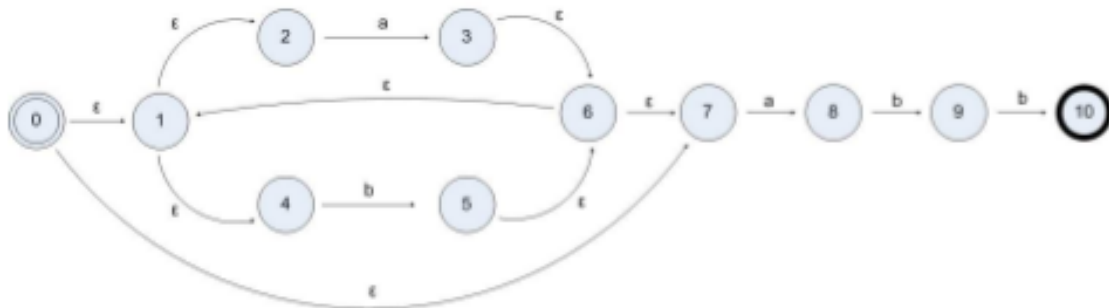
Calcul de $D = (Q', \Sigma, \delta', q'_0, F')$

- $q'_0 = \epsilon\text{-fermeture}(q_0)$
- $F' = \{E \mid E \in Q' \wedge \exists e \in E. e \in F\}$
- Construction de Q' et δ'
 1. On commence par q'_0
 2. Pour chaque $x \in \Sigma$, on ajoute $\epsilon\text{-fermeture}(\text{Transiter}(q'_0, x))$ dans Q'
 3. On continue jusqu'à stabilité

Exemple

Automate $N = (Q, \Sigma, \delta, q_0, F)$

$Q = \{0, 1, 2, 3, 4, 5, 6, 7, 8, 9, 10\}$, $\Sigma = \{a, b\}$, $q_0 = 0$ et $F = \{10\}$



Langage reconnu : $L(N) = (a|b)^*abb$

Calcul

1. $q'_0 = \epsilon\text{-fermeture}(q_0) = \epsilon\text{-fermeture}(0) = \{0, 1, 2, 4, 7\} = A$

2. $\epsilon\text{-fermeture}(\text{Transiter}(A, a)) =$
 $\epsilon\text{-fermeture}(\text{Transiter}(\{0, 1, 2, 4, 7\}, a)) =$
 $\epsilon\text{-fermeture}(\text{Transiter}(\{3, 8\})) = \{1, 2, 3, 4, 6, 7, 8\} = B$

3. $C = \epsilon\text{-fermeture}(\text{Transiter}(A, b)) = \{1, 2, 4, 5, 6, 7\}$

4. ...

$$A = \{0, 1, 2, 4, 7\} \qquad D = \{1, 2, 4, 5, 6, 7, 9\}$$

$$B = \{1, 2, 3, 4, 6, 7, 8\} \quad E = \{1, 2, 4, 5, 6, 7, 10\}$$

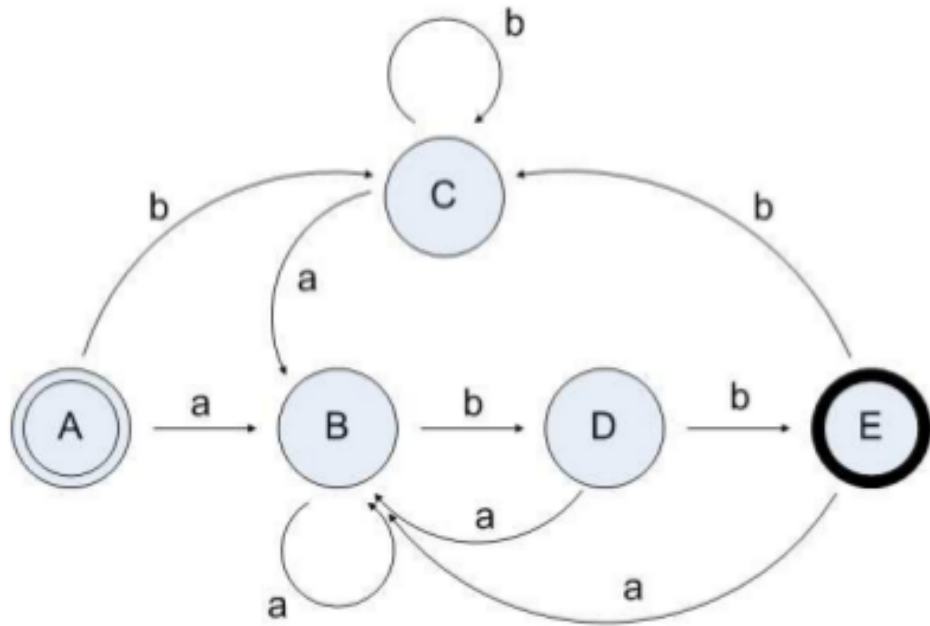
$$C = \{1, 2, 4, 5, 6, 7\}$$

5. $F' = \{E\}$

Résultat

Table de transition δ' pour D

	a	b
A	B	C
B	B	D
C	B	C
D	B	E
E	B	C



Minimisation du nombres d'états d'un DFA

- Initialisation
 1. On retire les états non atteignables
 2. On partitionne l'ensemble Q des états en deux classes, les états finaux et les états non finaux
- Principe d'éclatement de partition
 1. Prendre une classe C_i
 2. Lui appliquer une transition $x \in \Sigma$
 3. Séparer les éléments de C_i qui n'aboutissent pas à une même classe
 4. Continuer jusqu'à obtention d'un point fixe

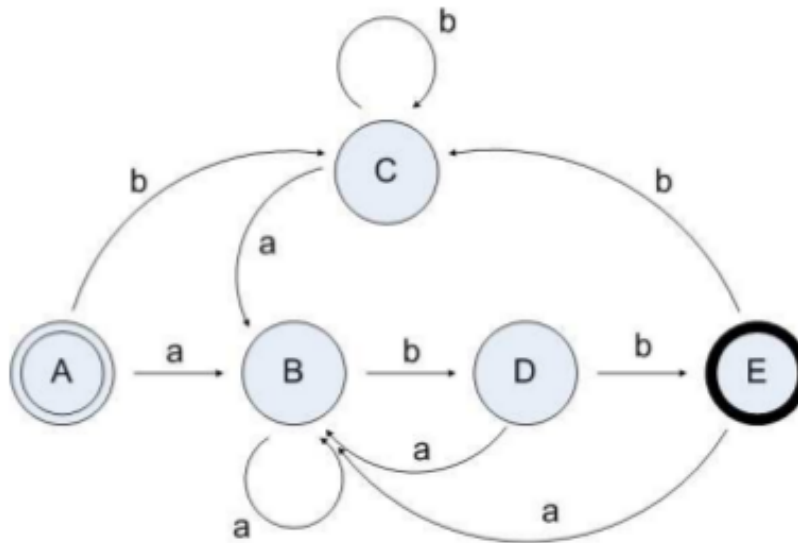
Définition d'un DFA minimal

Calcul de $A' = (Q', \Sigma, \delta', q'_0, F')$

- Choisir un représentant dans chaque classe C_i
- Les états de Q' sont les représentant choisis
- L'état q'_0 est le représentant de la classe contenant q_0
- L'ensemble F' est constitué des représentants de classes contenant un état de F
- Supprimer les états morts de Q'
- Supprimer de Q' tous les états non accessibles depuis l'état de départ

Exemple de minimisation

Automate A_5



Langage reconnu : $L(A_5) = (a|b)^*abb$

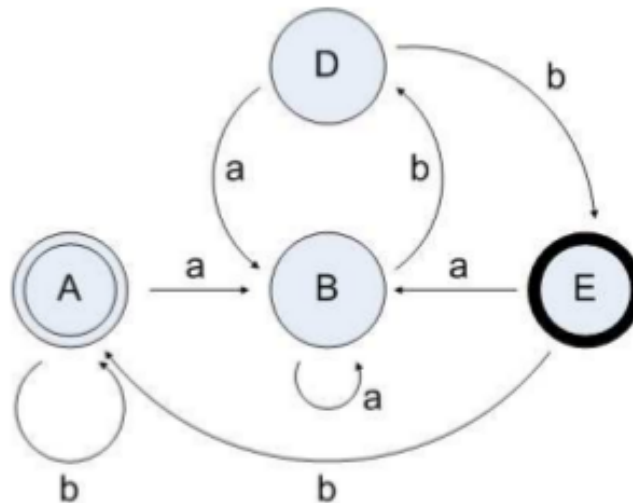
Calcul

1. $(E), (ABCD)$
2. $(ABC), (D), (E)$
3. $(AC), (B), (D), (E)$
4. On choisit respectivement A, B, D et E comme représentant de la classe $(AC), (B), (D)$ et (E)
5. Donc $Q' = \{A, B, D, E\}$
6. $q'_0 = A$ et $F' = \{E\}$

Automate A'

Table de transition de A'

	a	b
A	B	A
B	B	D
D	B	E
E	B	A



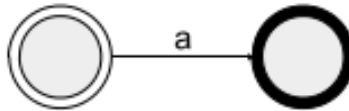
Il n'y a pas d'états morts dans Q' et tous les états sont accessibles depuis q'_0

Construction d'un AFN à partir d'une expression régulière (définition inductive)

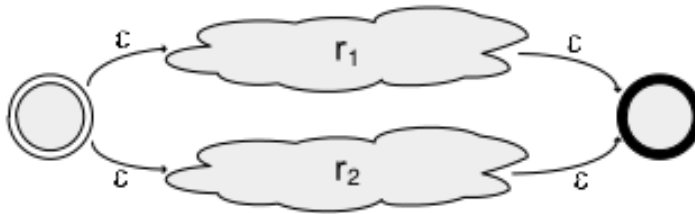
$r = \epsilon$



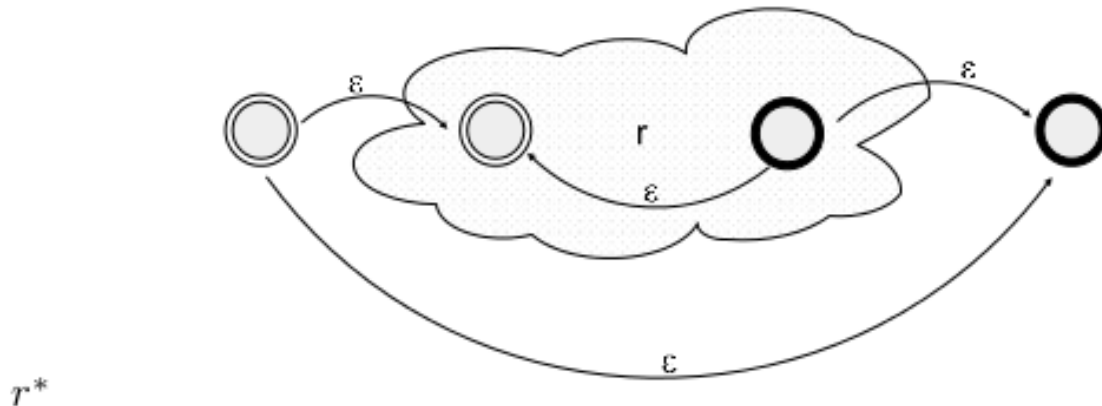
$a \in \Sigma$



$r_1 | r_2$



Construction d'un AFN à partir d'une expression régulière (définition inductive)



Développement d'un analyseur lexical

Un véritable analyseur lexical est

- très gros
- très complexe
- très optimisé

Il existe des outils qui, à partir de spécifications basées sur les expressions régulières, génèrent automatiquement l'analyseur lexical

- lex, flex en C, ocamllex en OCaml, JavaCC en Java, etc.

Ocamllex

- Schéma
 1. Chaque expression régulière est compilée en un automate
 2. L'ensemble des automates sont fusionnés en un seul
 3. L'automate résultant est déterminisé
 4. L'automate est minimisé
- Remarque : En fait, ocamllex produit directement des automates déterministes et ne les minimise pas

Analyseur lexical

Structure du fichier source

- **Entête** : Morceau de code du langage cible (Ocaml) qui sera mis en entête du fichier cible. Ce morceau de code peut contenir des commentaires et des définitions.
- **Corps** : Composé des expressions régulières associées aux actions sémantiques. Ces règles peuvent être précédées de définitions introduisant des noms pour des expressions régulières particulières. Les actions utilisent les primitives du langage cible.
- **Épilogue** : Il s'agit du code du langage cible qui est copié à la fin du fichier cible engendré. Ce code pourra contenir le programme principal utilisant la fonction d'analyse lexicale.

Expressions régulières étendues

- $[\hat{s}]$: reconnaît les caractères autres que ceux dans s , avec s désignant un ensemble de caractères
- $"w"$: reconnaît la chaîne de caractères w
- $r\{n\}$: désigne les mots formés de n occurrences de mots dans $L(r)$
- $r\{n,m\}$: désigne les mots formés de k occurrences de mots dans $L(r)$ avec $n \leq k \leq m$

Unité lexicale et Ocamllex

En Ocamllex, les unités lexicales (*token*) sont associées à leur valeur. Chaque token est vu comme un constructeur du type des tokens et prend en argument la valeur du token

Actions sémantiques (1)

- `Lexing.lexeme lexbuf`

Retourne la chaîne de caractères reconnue

- `Lexing.lexeme_char lexbuf n`

Retourne le $n^{\text{ème}}$ caractère de la chaîne reconnue. Le premier caractère correspond à $n = 0$.

- `Lexing.lexeme_start lexbuf`

Retourne la position absolue dans le texte d'entrée du début de la chaîne reconnue. Le premier caractère lu depuis le texte d'entrée est à la position 0.

Actions sémantiques (2)

- `Lexing.lexeme_end lexbuf`

Retourne la position absolue dans le texte d'entrée de la fin de la chaîne reconnue. Le premier caractère lu depuis le texte d'entrée est à la position 0.

- `entrypoint lexbuf`

(où *entrypoint* est le nom d'un autre point d'entrée dans la même définition de l'analyseur lexical). Appelle récursivement l'analyseur lexical sur le point d'entrée donnée. Très utile pour notamment les commentaires imbriqués.

Example 1 (1)

```
(* File exemple1/lexer.mll *)
{
  open Tokens          (* The type token is defined in tokens.mli *)
  exception Eof
}

rule token = parse
  [' ' '\t']          { token lexbuf }      (* skip blanks *)
| '\n'                { EDL }
| ['0'-'9']+          { INT(int_of_string (Lexing.lexeme lexbuf)) }
| '+'                 { PLUS }
| '-'                 { MINUS }
| '*'                 { TIMES }
| '/'                 { DIV }
| '('                 { LPAREN }
| ')'                 { RPAREN }
| eof                 { raise Eof }
```


Exemple 1 (2)

```
(* File exemple1/tokens.mli *)
```

```
type token =  
  | INT of int  
  | PLUS | MINUS | TIMES | DIV  
  | LPAREN | RPAREN  
  | EOL
```

Exemple 1 (3)

```
(* File exemple1/main.ml *)

let _ =
  let count = ref 0 in
  try
    let lexbuf = Lexing.from_channel stdin in (* Fabrication du flux *)
    while true do
      let _ = Lexer.token lexbuf in
      count := !count + 1
    done
  with Lexer.Eof ->
    Printf.printf "\nJ'ai reconnu %d lexèmes\n" !count;
    exit 0
```

Exemple 2 (lexer.mll)

```
(* File exexemple2/lexer.mll *)
{
  open Tokens          (* The type token is defined in tokens.mli *)
  exception Illegal_character
}

let ident = ['a'-'z' 'A'-'Z'] ['a'-'z' 'A'-'Z' '0'-'9' '_' ]*

rule token = parse
  [' ' '\t' '\n'] { token lexbuf }      (* skip blanks *)
| ['0'-'9']+      { INT(int_of_string (Lexing.lexeme lexbuf)) }
| '+'             { PLUS }
| '-'             { MINUS }
| '='             { EQUAL }
| eof             { EOF }
| "let"           { LET }
| "in"            { IN }
| ident           { IDENT(Lexing.lexeme lexbuf) }
| _               { raise Illegal_character }
```

Exemple 2 (tokens.mli)

```
(* File exemple2/tokens.mli *)
```

```
type token =  
  | INT of int  
  | IDENT of string  
  | PLUS | MINUS  
  | EQUAL  
  | LET | IN  
  | EOF
```

Exemple 2 (main.ml)

```
(* File exemple2/main.ml *)

open Tokens

let token2txt token =
  match token with
  | INT(x)   -> Printf.sprintf "INT(%d)" x
  | IDENT(s) -> Printf.sprintf "VAR(%s)" s
  | PLUS      -> "PLUS"
  | MINUS     -> "MINUS"
  | EQUAL     -> "EQUAL"
  | LET       -> "LET"
  | IN        -> "IN"
  | EOF       -> ""

let rec parse lexbuf l =
  let token = Lexer.token lexbuf in
  if (token = EOF)
  then List.rev l
  else parse lexbuf ((token2txt token)::l)

let _ =
  let l = ref [] in
  let lexbuf = Lexing.from_channel stdin in (* Fabrication du flux *)
  let l = parse lexbuf [] in
  Printf.printf "Entrée: %s\n" (String.concat " " l)
```

Ambiguïtés

- Problème : il existe des ambiguïtés :
 - "let" pourrait être reconnu comme un identificateur
 - "lettre" pourrait aussi être reconnu comme la séquence
LET; IDENT("tre") ou encore
IDENT("let"); IDENT("tre")
- Solution : des règles de priorité :
 - Entre deux lexèmes de taille différentes : choisir le plus long.
 - Sinon, suivre l'ordre de définition des lexèmes.

Gestion des Commentaires

Elimination des commentaires

- Objectif

Supprimer les commentaires dès l'analyse lexicale. Ainsi, les commentaires n'ont aucun impact sur toutes les phases suivantes du compilateur.

- Différents styles de commentaires

- `// Commentaires à la C++`
- `/* Commentaires à la C
sur deux lignes */`
- `(* Commentaires imbriqués (* à la Caml *) *)`

Commentaires à la C++

Les commentaires s'étendent d'un mot particulier jusqu'à la fin de la ligne. Il est facile d'éliminer ce type de commentaire en ajoutant une règle dans l'analyseur `token` :

```
| "//" [^'\n']* '\n'?      { token lexbuf }
```

Remarques

- Attention de ne pas utiliser `_*` à la place de `[^'\n']` sinon les commentaires s'étendraient jusqu'au dernier retour à la ligne.
- L'utilisation de `'\n'?` permet de traiter le cas d'un commentaire sur la dernière ligne.

Commentaires à la C

Les commentaires sont compris entre deux mots particuliers, mais il ne peuvent pas être imbriqués. Il n'est pas simple de trouver une expression régulière décrivant ce type de commentaires.

La solution consiste à recourir à un deuxième automate `incomment`, lancé à l'ouverture du commentaire et chargé de reconnaître la fin du commentaire.

Automates pour les commentaires à la C

```
rule token = parse
    ...
    | "/*"      { incomment lexbuf }

and incomment = parse
    | "*/"      { token lexbuf }
    | _         { incomment lexbuf }
    | eof       { raise Unterminated_comment }
```

Commentaires à la Caml

- Les commentaires sont compris entre deux mots particuliers et peuvent être imbriqués. Ce dernier type de commentaire est utile pour commenter du code source qui contient lui même des commentaires.
- Il n'est pas possible d'utiliser un deuxième automate car le niveau d'imbrication des commentaires est arbitraire. Il n'existe pas d'expression régulière reconnaissant ce type de commentaires. Pour s'en sortir, on ajoute de l'état aux automates.

Automate pour les commentaires à la Caml

```
{
  let depth = ref 0
}

rule token = parse
  ...
  | "("      { depth := 1; incomment lexbuf }

and incomment = parse
  | ")"
    { depth := !depth - 1;
      if !depth <= 0 then
        token lexbuf          (* Premier niveau, on ferme le premier "(" *)
      else
        incomment lexbuf }    (* on ferme un autre "(" *)
  | "("
    { depth := !depth + 1;
      incomment lexbuf }
  | _      { incomment lexbuf }
  | eof    { raise Unterminated_comment }
```

Récupérer une chaîne (1)

- Objectif

Récupérer le contenu d'une chaîne de caractères entourés par le caractère ". On ajoute un lexème `STRING` of `string` dans le type des lexèmes.

- Solution naïve

```
rule token = parse
  ...
  | "\"" [^"]*" * "\""
    { let lxm = Lexing.lexeme lexbuf in (* lxm est la suite reconnue *)
      (* Supprimer le premier et le dernier caractère de lxm *)
      STRING (String.sub lxm 1 (String.length lxm - 2)) }
```

Récupérer une chaîne (2)

- Problème

Il peut être légitime de vouloir mettre une double quote dans le corps de la chaîne de caractères à reconnaître.

- Solution

Utiliser deux automates et un mécanisme de citation (`\` pour `"` et `\\` pour `\`).

Récupérer une chaîne (automate)

```
{
  let sbuff = Buffer.create 16    (* fabriquer le buffer *)
}

rule token = parse
  ...
  | '"'    { STRING(instring lexbuf) }

and instring = parse
  | '"'    { let s = Buffer.contents sbuff in          (* fin de chaîne *)
              Buffer.clear sbuff;
              s }
  | '\\\'  ('"'|'\\\'')
              { let c = Lexing.lexeme_char lexbuf 1 in  (* caractère spécial *)
                Buffer.add_char sbuff c;
                instring lexbuf }
  | _      { let c = Lexing.lexeme_char lexbuf 0 in    (* caractère reconnu *)
              Buffer.add_char sbuff c;
              instring lexbuf }
  | eof    { raise Unterminated_string }
```